

VIS iGEM 2026 · Team 6423 · Cadture

FOR STUDENTS

學生講義

14 次 × 3 小時 · Wiki 製作課程

內容基準：<https://2026.igem.wiki/vis/>

線上課堂：[/class](#)

A4 · 建議雙面列印

一、課表與常用檢查

1. 一頁課表

課程名稱：做我們的 iGEM Team Wiki (Cadture)

時間：14 次 × 3 小時

1. 架環境、認識 wiki
2. HTML
3. CSS 與風格
4. Markdown 改內容
5. Git 與誠信
6. 網址與 Next 資料夾
7. 選單與頁尾
8. 元件與內容怎麼變成網頁
9. 首頁資料
10. 設計與手機版
11. 自己加一頁
12. 圖、英文、好讀
13. 檢查表與上線概念
14. 發表會

你會一點 JS：物件 / 陣列會在選單與列表裡一直出現；元件≈會 return 畫面的 function。

你每天最少會的技能：改 content 裡的檔 → 存檔 → 重新整理瀏覽器。

鐵律：英文上站、不造假、路徑不亂改、照片要同意。

2. 品質 / iGEM 檢查表（第 13 次與期末用）

A. 內容

- ☐ 英文為主，專有名詞一致 (Cadture、Pb、Cd)
- ☐ 無未經確認的實驗數據
- ☐ Description / Engineering / HP / Contribution / Team 有人負責
- ☐ 圖片有說明與來源

B. 結構

- ☐ 導覽連結皆可點
- ☐ 重要路徑未隨意改名
- ☐ 新增頁有 md + page + nav

C. 技術

- ☐ `npm run dev` 正常
- ☐ `npm run build` 通過（或已知錯誤有紀錄）
- ☐ 無明顯破版（手機寬度粗查）

D. 競賽意識

- ☐ 知道 freeze 後不能大改
- ☐ 知道正式托管與本機預覽不同
- ☐ 知道 Attributions 要用官方格式

3. JS 與本站對照

概念	在 Cadture 專案
物件 { a: 1 }	nav.ts 一筆選單、md front matter
陣列 [...]	children、home 的 problems/solution
.map	首頁列表渲染
function + return	page.tsx、各 Component
參數	props (如 slug="description")
import / export	跨檔使用 WikiPage、圖示
字串	網址 path、title、錯誤訊息
除錯	終端機紅字、漏逗號、路徑不一致

二、學習地圖與操作基礎

課程總覽：我們要學會『做出這個網站』

1. 這門課到底在訓練什麼？

1.1 請老師先對學生說（建議照念或改寫）

我們不是只學「怎麼改幾個英文字」。
我們要搞懂：這個 *Cadture Wiki* 是怎麼做出來的。
結束時你要能：啟動專案、改內容、加一頁、解釋程式怎麼把 *Markdown* 變成網頁，而且不造假數據。

1.2 錯誤期待 vs 正確期待（投影討論 10 分）

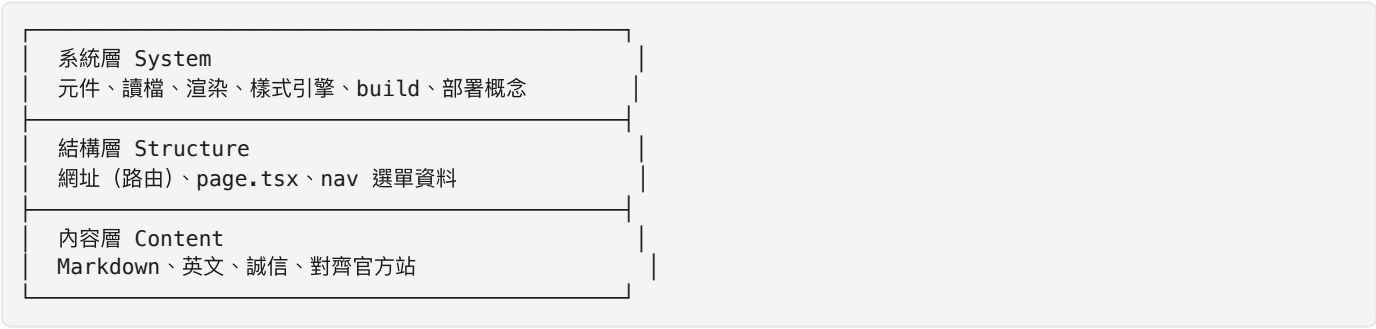
請學生兩兩討論後分享：

不夠的狀態	本課程要的狀態
會打開 md 改一行	知道 md 被哪個程式讀走
會複製別人的 page.tsx	能解釋每一行在幹嘛
本機好看就好	會跑 build、會檢查連結
頁面很多就好	路徑穩定、內容誠實

停下來確認（老師問全班）：

「如果只會改 md，算不算這門課及格？」→ 引導到：不夠，還要結構與系統。

2. 三層能力模型（請學生畫在筆記本正中央）



2.1 對照練習（15 分，務必做）

老師報任務，學生舉牌或口頭答「碰哪一層」：

任務	答案
改 Description 某段英文	內容
讓 /hello 這個網址存在	結構
解釋 WikiPage 做什麼	系統
選單多一個項目	結構 (nav 資料)
改首頁三張 Problem 卡文案	內容 (home.md)
改卡片圓角	系統 / 表現 (class)
git commit	工程流程 (跨層)

2.2 對應檔案 (請學生抄表)

層	主要路徑	第幾次會深挖
內容	content/home.md 、 content/pages/*.md	4、9、12
結構	src/app/**/page.tsx 、 src/data/nav.ts	6、7、11
系統	src/lib/content.ts 、 WikiPage 、 MarkdownBody 、 layout	8、9、10、13

3. 本站是怎麼做出來的？ (全貌講解 15—20 分)

3.1 技術零件表

技術	角色	學生要會到什麼程度
Next.js App Router	資料夾對應網址	會對、會新增 page
React 元件	函式回傳畫面	會讀、會改 props 小字
TypeScript	型別提示	會看、不要求寫複雜 type
Tailwind	class 控樣式	會猜、會改間距顏色
Markdown + gray-matter	內容與 front matter	會編、會修壞掉的 ---
react-markdown	md→HTML	知道它在管線裡

3.2 一次請求完整故事 (請慢講、可講兩遍)

以使用者開啟 `/engineering`：

1. 瀏覽器向本機伺服器要 `/engineering`
2. Next 找到 `src/app/engineering/page.tsx`
3. 執行裡面的 `Page` 函式元件
4. 它 return：`<WikiPage slug="engineering" />`
5. `WikiPage` 呼叫 `getPageBySlug("engineering")`
6. 程式讀取檔案 `content/pages/engineering.md`

7. 用 gray-matter 拆成：

- frontmatter (title、subtitle...)
- body (正文 Markdown 字串)

8. PageHero 用 frontmatter 畫大標區

9. MarkdownBody 把 body 變成真正的標題、段落、表格

10. 外層 layout.tsx 早已包好 Navbar、Footer

11. 完整頁面出現在瀏覽器

停下來確認：抽 3 位學生，一人講 1-4 步、一人 5-8、一人 9-11。

3.3 首頁為什麼不一樣？（5-8 分）

頁種類	內容從哪來	畫面誰排
文件頁 (如 /team)	一篇 pages/*.md	WikiPage
首頁 /	home.md 的 YAML 清單	src/app/page.tsx 自己 .map

首頁順序（對齊官方 <https://2026.igem.wiki/vis/>）：

1. The Problem
2. Our Solution
3. Project Highlights
4. Towards a Sustainable Future
5. Human Practices
6. Engineering Cycle
7. Meet Our Team

4. 14 次學習路徑（帶讀 10 分）

階段	次	你要帶走的能力
奠基	1	跑起專案、三層模型、官方對照
	2	HTML/DOM 樹、JS 改節點
	3	CSS token、class 狀態→接 Tailwind
內容	4	md 管線 + 追到程式
	5	Git 工作流 + 誠信
系統	6	路由與 page.tsx
	7	nav 資料驅動
	8	元件引擎精讀（高點）
	9	首頁組裝
打造	10	設計系統
	11	總裝一頁（硬驗收）
	12	產品品質
	13	build 與交付
	14	發表：證明會做

5. 學期過關標準（印給學生）

結束時你必須獨立做到：

1. 啟動 `npm run dev` 並說明三個資料夾
2. 修改一頁 `md` 並驗證
3. 新增一頁（`md` + `page` + `nav`）並解釋為何三步
4. 在程式裡指認：`page.tsx`、`nav.ts`、`getPageBySlug`、`WikiPage`
5. 口述 `/engineering` 請求流程（可看自己畫的圖）
6. 說明首頁 `YAML` 與 `.map`
7. 跑過 `npm run build`
8. 遵守誠信與官方對齊

只會第 2 項 → 未過關。

6. 內容層速查（改文字時用）

網址	檔案
<code>/</code>	<code>content/home.md</code>
<code>/description</code>	<code>content/pages/description.md</code>
其他內頁	<code>content/pages/<同名>.md</code>

```
npm run dev
# 改檔 → 存檔 → 重新整理
```

逐步細節見 第 4 次 教材。

7. 老師如何避免「講太快」

1. 每個「停下來確認」真的停 1—2 分鐘。
2. 每個完整步驟至少抽 2 組螢幕給全班看。
3. 寧可少講下一單元，也要讓 80% 學生做完檢查表。
4. 進度快的人做「加分 / 除錯劇本」，不要直接往下趕課。
5. 第 6、8、11 次預設會超時——後面各留彈性，或壓縮第 3 次展示時間。

8. 相關連結

- 課表：`/class`
- 第 1 次：`/class/session-01`
- 老師詳細備課：`docs/curriculum/LESSON_PLAN_14.md`

三、第 1–14 次課堂講義

每堂含：學習目標、分鐘表、完整步驟、完成檢查表、作業。

第 1 次 | 專案啟動與網站全貌

跑起來、看懂目錄、建立三層心智模型（放慢版）

時數：3 小時

本堂學習目標

- 成功啟動專案並排除至少一種常見錯誤
- 畫出內容 / 結構 / 系統三層圖
- 對照官方站完成七段首頁檢查表
- 用 JS 物件描述一筆選單資料

0. 分鐘表（請投影）

分鐘	做什麼	產出
0—15	破冰：你心目中「會做網站」是什麼	便利貼 1 張
15—35	課程目標與三層模型導讀	筆記本中央圖
35—75	安裝與啟動（巡堂）	localhost 截圖
75—85	休息	
85—120	官方站 vs 本站首頁七段	對照表填完
120—150	資料夾地圖導覽 + 問答	樹狀圖
150—170	JS 物件練習	小程式可跑
170—180	檢查表、作業	勾完

1. 開場討論（15 分）— 老師引導語

問題 1：做一個「會改文字的網頁」和「理解網站怎麼做」差在哪？

問題 2：如果只有一個人會改 code，iGEM 衝刺期會發生什麼事？

請 2—3 組分享。老師總結：本課要培養可交接的建造能力。

2. 我們要做的網站是什麼？（10 分）

2.1 專案事實（請學生抄）

項目	內容
隊伍	VIS iGEM 2026 · 6423
專案名	Capture
主題	Pb、Cd 與廢水
官方站	https://2026.igem.wiki/vis/
本專案	可教學、可維護的實作站

2.2 兩個網址

	官方站	本機站
網址	2026.igem.wiki/vis	localhost:3000
角色	內容基準、線上版	學習建造與日常改稿
今天	對照	必須跑起來

停下來確認：抽問「內容要以誰為準？」→ 官方 + 隊上確認。

3. 完整步驟：啟動專案（放慢，逐步勾）

步驟 1 | 檢查 Node（每人）

```
node -v  
npm -v
```

- 成功：出現 `v18 / v20 / v22` 之類
- 失敗：安裝 Node LTS → 關掉終端機重開 → 再打一次

老師巡堂：看到紅字先別急著幫打指令，讓學生念出錯誤第一行。

步驟 2 | 進入專案根目錄

```
pwd  
ls
```

必須看得到 `package.json`、`src`、`content`。

若沒有：`cd` 進錯層了。

步驟 3 | 安裝依賴

```
npm install
```

期間任務（寫在黑板上）：

打開官方站，開始填第 4 節對照表前半。

步驟 4 | 啟動開發伺服器

```
npm run dev
```

看到 `Local: http://localhost:3000` 之類訊息。

步驟 5 | 瀏覽器驗證清單

請學生逐項打勾：

- ☐ 標題有 Cadture
- ☐ 有 The Problem 區塊
- ☐ 有 Our Solution 五步
- ☐ 上方選單有 Project / Wet Lab 等
- ☐ 有 Course 可進教材

步驟 6 | 失敗劇本演練（全班一起看一種）

老師可示範或口述：

錯誤訊息關鍵字	原因	學生動作
command not found	沒裝 Node / PATH	重裝、重開終端
ENOENT package.json	目錄錯	cd 到有 package.json
EADDRINUSE 3000	port 占用	換 3001 或關掉舊 process
install 權限	環境限制	換路徑、問老師

4. 官方站對照（放慢 35 分）

4.1 並排視窗

左：官方 <https://2026.igem.wiki/vis/>

右：localhost:3000

4.2 七段順序（一起念兩次）

- 1. The Problem
- 2. Our Solution
- 3. Project Highlights
- 4. Towards a Sustainable Future
- 5. Human Practices
- 6. Engineering Cycle
- 7. Meet Our Team

4.3 填表（每人）

#	區塊	官方有無	本機有無	一句英文關鍵句（抄）
1	Problem			
2	Solution			
3	Highlights			
4	Sustainable			
5	HP			
6	Engineering			
7	Team			

4.4 討論題（5 分）

「本機視覺比較可愛，能不能改掉官方的 Solution 五步名稱？」

引導：可以討論 UI，但故事骨架與宣稱要對齊 / 經確認。

5. 資料夾地圖（逐層，25 分）

老師每開一層就問：

5.1 content/

這裡全刪會怎樣？→ 頁面沒字或爆錯，但路由殼可能還在。

5.2 src/app/

這裡的資料夾名跟網址什麼關係？→ 幾乎一一對應。

5.3 src/components/

像樂高積木；Navbar 壞了會怎樣？→ 可能全站沒選單。

5.4 src/data/nav.ts

這是資料不是畫面；改這裡選單文字會變。

5.5 src/lib/content.ts

讀 md 的橋樑——第 8 次會拆。

5.6 學生作業：畫樹

限時 8 分，畫出至少：

```
iGemSite
├── content
├── src/app
├── src/components
└── src/data
```

並在旁邊標：內容 / 結構 / 系統。

6. JS 暖身：選單物件（20–25 分）

6.1 為什麼現在練？

因為第 7 次要改真的 `nav.ts`，結構一模一樣。

6.2 完整程式（請學生手打）

`practice/session01-nav.js`：

```
// 一筆選單 = 一個物件
const descriptionItem = {
  label: "Description",
  href: "/description",
  desc: "Background and problem",
};

// 多筆 = 陣列
const projectChildren = [
  descriptionItem,
  {
    label: "Engineering",
    href: "/engineering",
    desc: "Design → Build → Test → Learn",
  },
  {
    label: "Results",
    href: "/results",
    desc: "Experimental results",
  },
];

// 掃過每一筆 (之後 React 的 map 很像這個)
for (let i = 0; i < projectChildren.length; i++) {
  const item = projectChildren[i];
  console.log(i + 1, item.label, "→", item.href);
}

// 進階一點：用 for...of
console.log("---");
for (const item of projectChildren) {
  console.log(item.label + ": " + item.desc);
}
```

```
node practice/session01-nav.js
```

6.3 延伸（進度快的人）

把 `projectChildren` 改成函式 `printMenu(items)` 包起來再呼叫。

6.4 停下來確認

抽問：`href` 是什麼？少了會怎樣？`desc` 給誰看？

7. 本堂完成檢查表（下課前 10 分互查）

- ☐ node/npm 有版本
- ☐ 首頁截圖已交
- ☐ 七段對照表填完
- ☐ 三層圖有畫
- ☐ nav 小程式有跑出輸出
- ☐ 能用一句話說 content 與 app 差別

8. 回家作業（寫清楚截止）

1. 必做：150 字：「這個網站不是單一 HTML 檔，而是……」
2. 必做：背七段首頁順序（下堂抽背）
3. 必做：讀 `/class/how-to-edit` 課程總覽

4. 選做：為 5 個內頁寫「網址 ↔ 你猜的資料夾」

9. 下一堂預告（留 2 分）

我們會做出「會動的 HTML 樹」，因為 React 元件本質是在描述那棵樹。

第 2 次 | 網頁的結構：HTML 與 DOM

畫面是一棵樹——為理解 React / Next 打底（放慢版）

時數：3 小時

本堂學習目標

- 手刻完整 HTML 並解釋每個主要標籤
- 畫出 DOM 樹並指出父子關係
- 用 JS 完成改文字、新增節點、讀取列表
- 對照正式站：為何不把長文寫死在 HTML

0. 分鐘表

分鐘	活動
0—10	抽背首頁七段 + 本堂目標
10—50	手刻 HTML（分段檢查）
50—60	休息
60—90	畫 DOM 樹 + 互批
90—140	JS 三段操作
140—165	對照 Cadture 正式站討論
165—180	檢查表與作業

1. 連結上堂（5 分）

問：`content/` 是哪一層？`src/app/` 呢？

再問：今天的 HTML 練習主要對應「瀏覽器裡的什麼？」→ DOM 樹 / 結構。

2. 建立練習資料夾（完整步驟）

步驟 1

在磁碟建立：

```
practice/session-02/
```

步驟 2

稍後會有三個檔：

```
index.html
main.js
styles.css ← 可下堂再補，本堂可不作 css
```

步驟 3

用 VS Code「打開資料夾」指到 `session-02`，避免存錯地方。

3. HTML 分段建造 (40 分)

階段 A | 最小可開檔 (10 分)

先只做到 body 有 h1，就用瀏覽器打開確認：

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Cadture Structure Practice</title>
  </head>
  <body>
    <h1 id="site-title">Cadture</h1>
  </body>
</html>
```

檢查點 A： 雙擊開得起來、看得到 Cadture。

階段 B | 補齊骨架 (15 分)

加入 header、nav、main、footer。老師說明語意：不是為了好看，是為了結構可讀、之後好對應元件名稱。

階段 C | 放入官方故事的小樣本 (15 分)

Problem 三點、Solution 五步（英文用官方句，加深印象）。

完整參考（學生可對照，但鼓勵先自己打）：


```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Cadture Structure Practice</title>
  </head>
  <body>
    <header>
      <h1 id="site-title">Cadture</h1>
      <p id="tagline">Structure practice for building the real wiki</p>
    </header>

    <nav aria-label="Practice">
      <a href="#problem">Problem</a> |
      <a href="#solution">Solution</a>
    </nav>

    <main>
      <section id="problem">
        <h2>The Problem</h2>
        <ul id="problem-list">
          <li data-key="pb">Lead Pollution</li>
          <li data-key="cd">Cadmium Threat</li>
          <li data-key="water">Wastewater Challenge</li>
        </ul>
      </section>

      <section id="solution">
        <h2>Our Solution</h2>
        <ol id="solution-list">
          <li>Detection</li>
          <li>Capture</li>
          <li>Sequestration</li>
          <li>Removal</li>
          <li>Clean Water</li>
        </ol>
      </section>

      <button type="button" id="btn-title">Rename title</button>
      <button type="button" id="btn-add">Add note node</button>
      <button type="button" id="btn-read">Read solution steps to console</button>
      <div id="note-area"></div>
    </main>

    <footer>
      <small>Practice DOM – foundation for understanding React components.</small>
    </footer>

    <script src="main.js"></script>
  </body>
</html>

```

檢查點 C：不靠 JS 也能完整閱讀問題與解法列表。

4. DOM 樹作業（30 分，可放慢）

4.1 示範畫法

老師在白板上從 `body` 往下畫 3 層。

4.2 學生必交

畫出包含：`header`、`main`、`#solution-list` 與其下 5 個 `li`。

4.3 互批問題（兩人一組）

1. #btn-add 的父節點是？
 2. 第一個 section 的前一個兄弟大致是？
 3. 若把 script 放到 head 且不延遲，可能有什麼問題？
-

5. JS 三段式（分段做，每段驗收）

建立 main.js 。

練習 1 | 改文字（10 分）

```
const titleEl = document.querySelector("#site-title");
const btnTitle = document.querySelector("#btn-title");

console.log("titleEl =", titleEl);

btnTitle.addEventListener("click", function () {
  titleEl.textContent = "Cadture (edited via DOM)";
});
```

驗收：Console 不是 null；按鈕可改標題。

練習 2 | 新增節點（10 分）

```
const btnAdd = document.querySelector("#btn-add");
const noteArea = document.querySelector("#note-area");

btnAdd.addEventListener("click", function () {
  const p = document.createElement("p");
  p.textContent =
    "In the real Cadture site, long text usually lives in Markdown files under content/.";
  noteArea.appendChild(p);
});
```

驗收：每按一次多一段（可討論要不要先清空）。

練習 3 | 讀列表到陣列（15 分）

```
const btnRead = document.querySelector("#btn-read");
const solutionList = document.querySelector("#solution-list");

btnRead.addEventListener("click", function () {
  const steps = [];
  const items = solutionList.querySelectorAll("li");
  items.forEach(function (li) {
    steps.push(li.textContent.trim());
  });
  console.log("Solution steps array:", steps);
  console.log("Length =", steps.length);
  alert("Read " + steps.length + " steps. Check console.");
});
```

驗收：Console 有 5 個字串。

連結下下堂：這就像把資料拿出來；正式站是資料先存在 md，再被 map 成畫面。

6. 對照正式站（20 分討論 + 筆記）

打開 localhost Cadture 首頁。

練習頁	正式站
步驟寫死在 HTML	步驟在 <code>content/home.md</code> 的 <code>solution</code>
自己 <code>appendChild</code>	React 依資料重新渲染
單檔	多檔分層協作

書面題（交）：用 80–120 字說明「為什麼正式站不把全部英文寫死在一個 HTML？」

7. 完成檢查表

- ☐ HTML 三階段都通過
- ☐ DOM 樹有畫且互批
- ☐ JS 三練習都成功
- ☐ 書面對照題完成

8. 回家

1. 補畫「你猜的」 `/description` 頁結構樹
2. 下堂會對到 `PageHero`、文章區等真實元件名稱

9. 下一堂

CSS 變數與 class 狀態——設計系統的最小版，接到 Tailwind。

第 3 次 | 樣式系統入門：CSS 到 className

從設計 token、class 狀態，接到本站 Tailwind（放慢版）

時數：3 小時

本堂學習目標

- 用 CSS 變數做出可維護的主色與卡片
- 用 class 表達狀態 (is-active)
- 對照 globals.css 與 page.tsx 的 class 寫法
- 完成 RWD 粗檢查

0. 分鐘表

分鐘	活動
0—10	回顧 DOM；本堂在三層模型的位置（表現 / 系統）
10—55	寫 styles.css + 套到 session-02
55—65	休息
65—100	class 狀態 + JS
100—135	對照本站 globals / page.tsx 拆 class
135—165	窄螢幕檢查 + 小挑戰
165—180	檢查表、作業

1. 為什麼建造網站一定要有「樣式系統」？

1.1 討論（8 分）

若 10 個人各自選顏色：

- 首頁粉紅、Description 螢光綠、Team 深藍……訪客會覺得？
- iGEM 評審快速掃站時，混亂風格會怎樣？

結論：樣式是產品一致性，不是裝飾而已。

1.2 本站品牌（請抄）

Token	色碼	用途
background	#FFF8F6	頁面底
primary	#C94C67	重點、按鈕
primary-dark	#B43852	標題
border	#F0C4CE	卡片邊

2. 完整步驟：為 session-02 加上 styles.css

步驟 1 | 建立檔案

路徑：practice/session-02/styles.css（與 index.html 同層）

步驟 2 | 貼上完整 CSS (分段理解)

第一段：變數 (token)

```
:root {
  --bg: #fff8f6;
  --ink: #4a2f34;
  --primary: #c94c67;
  --primary-dark: #b43852;
  --border: #f0c4ce;
  --card: #ffffff;
}
```

停下來確認：問「若只改 `--primary`，哪些地方應跟著變？」

第二段：body 基礎

```
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, -apple-system, sans-serif;
  background: var(--bg);
  color: var(--ink);
  line-height: 1.55;
}
```

第三段：版面寬度

```
header, nav, main, footer {
  max-width: 960px;
  margin: 0 auto;
  padding: 1rem 1.25rem;
}

h1 { color: var(--primary-dark); }
h2 { color: var(--primary-dark); }
```

第四段：卡片感列表

```
#problem-list,
#solution-list {
  background: var(--card);
  border: 2px solid var(--border);
  border-radius: 1.25rem;
  padding: 1rem 1.25rem;
  box-shadow: 3px 3px 0 rgba(201, 76, 103, 0.08);
}

#problem-list li {
  padding: 0.35rem 0.25rem;
  border-radius: 0.5rem;
}
```

第五段：按鈕

```
button {
  border: 2px solid var(--primary-dark);
  background: var(--primary);
  color: #fff;
  font-weight: 700;
  border-radius: 999px;
  padding: 0.55rem 1rem;
  margin: 0.35rem 0.35rem 0.35rem 0;
  cursor: pointer;
}

button:hover {
  background: var(--primary-dark);
}
```

第六段：狀態

```
.is-active {
  outline: 3px solid var(--primary);
  background: #fff5f8;
}
```

步驟 3 | HTML 引入

在 `<head>` 加：

```
<link rel="stylesheet" href="styles.css" />
```

步驟 4 | 重新整理驗證

- ☐ 背景暖白
- ☐ 列表像卡片
- ☐ 按鈕圓角粉色

步驟 5 | token 實驗 (10 分)

把 `--primary` 暫時改成 `#2f6f4e`，重新整理，觀察哪些變了，再改回玫瑰主色。
寫一句話：為什麼要用變數而不是到處寫死色碼？

3. 完整步驟：class 狀態 + JS

步驟 1 | 更新 main.js 片段

```
const problemItems = document.querySelectorAll("#problem-list li");

problemItems.forEach(function (li) {
  li.style.cursor = "pointer";
  li.addEventListener("click", function () {
    problemItems.forEach(function (other) {
      other.classList.remove("is-active");
    });
    li.classList.add("is-active");
    console.log("Activated:", li.textContent.trim());
  });
});
```

步驟 2 | 操作驗證

點 Lead → 只有 Lead 高亮；再點 Cadmium → 高亮轉移。

步驟 3 | 觀念對照（老師慢慢講）

你做的	本站以後
<code>classList.add("is-active")</code>	依狀態拼 <code>className</code>
CSS <code>.is-active</code>	預先寫好狀態樣式
一次只亮一個	單一選取 UI 邏輯

4. 對照本站（放慢 35 分）

4.1 打開 `src/app/globals.css`

找到 `:root` 與 `--primary`。

與自己的 `styles.css` 並排，圈出相似處。

4.2 打開 `src/app/page.tsx`

搜尋 `className=`，挑一行最長的抄下，拆成表格：

class	我猜的意思	問過同學/老師後
...至少 8 個		

4.3 常見 Tailwind 速查（本堂夠用）

class	意思
<code>mt-4</code>	上外距
<code>p-6</code>	內距
<code>flex</code>	flex 配置
<code>grid</code>	網格
<code>gap-4</code>	子元素間距
<code>text-center</code>	置中
<code>font-bold</code>	粗體
<code>rounded-2xl</code>	大圓角
<code>border-2</code>	邊框
<code>md:grid-cols-3</code>	中等寬度以上 3 欄
<code>text-primary</code>	使用主題主色（本站）

4.4 停下來確認

抽問：`md:grid-cols-3` 在手機上通常是幾欄？為什麼？

5. 窄螢幕檢查（15 分）

1. 打開 localhost 首頁
2. DevTools → 裝置工具列
3. 選 iPhone 寬度
4. 記錄：

檢查	OK ?	備註
Problem 卡片可讀		
選單可開		
按鈕可點		
文字會不會溢出		

6. 完成檢查表

- ☐ CSS 六段都理解並套用
- ☐ token 實驗有做並改回
- ☐ is-active 可演示
- ☐ 拆過至少 8 個本站 class
- ☐ 窄螢幕表填完

7. 回家作業

1. 練習頁 vs 本站截圖各 1，寫相似 3、相異 3
2. 讀 `/class/how-to-edit` 的「一次請求故事」預習
3. (選做) 在 `Card.tsx` 圈 5 個 class

8. 下一堂

內容層：Markdown 是系統的輸入；要追到誰讀它。

第 4 次 | 內容層：Markdown 管線實作

改 md、驗證畫面、追到程式——放慢到每位同學都追得到

時數：3 小時

本堂學習目標

- 獨立完成 front matter 與正文修改
- 能追蹤 md → content.ts → WikiPage → 畫面
- 會更新 toc 與 ## 標題 id 對應
- 能處理至少兩種『畫面沒變 / 頁面壞掉』狀況

0. 分鐘表

分鐘	活動
0—10	抽背首頁七段；本堂目標
10—25	管線圖導讀（先講再做）
25—55	熱身：改 subtitle
55—65	休息
65—110	改正文 + toc
110—145	追 code 三站
145—170	除錯關卡 + 互查
170—180	作業

1. 先講清楚：你改的是「輸入」

作者編輯 md → (輸入) → 讀檔函式 → 元件組裝 → 瀏覽器

Markdown 不是整個網站；它是內容層資料格式。

2. 課前環境檢查（5 分）

```
cd 你的/iGemSite
npm run dev
```

- ☐ 終端機 Ready
- ☐ 能開 /description

3. 完整步驟 A | 改 front matter（放慢）

以 content/pages/description.md 為例（其他頁同理）。

步驟 A1 | 打開檔案

VS Code 檔案樹：content → pages → description.md

步驟 A2 | 認清兩段結構

- 1. 第 1 個 `---` 到第 2 個 `---`：設定 (YAML)
- 2. 之後：正文 (Markdown)

步驟 A3 | 只改 subtitle

找到：

```
subtitle:
```

改成你的英文（練習可暫用）：

```
subtitle: Why our team cares about Pb and Cd in wastewater.
```

步驟 A4 | 存檔

確認編輯器沒有未存檔圓點。

步驟 A5 | 驗證

- 1. 開 `http://localhost:3000/description`
- 2. 重新整理
- 3. 看大標下方副標

步驟 A6 | 鄰座互查（3 分）

對方必須用手指著螢幕上「新副標」的位置。

若沒變——逐步排查（投影這張表）

順序	檢查
1	存檔了嗎？
2	網址是 <code>/description</code> 嗎？
3	<code>npm run dev</code> 還在嗎？
4	改的是 <code>description.md</code> 不是別頁？
5	YAML 的 <code>subtitle:</code> 縮排是否與其他鍵對齊？
6	是否少了 <code>---</code> ？

4. 完整步驟 B | 改正文並加標題

步驟 B1 | 在第二個 `---` 下方找到

```
## What Should this Page Contain?
```

步驟 B2 | 在其下插入（示範，可改成你的真實計畫）

```
## Our drafting notes
```

We are preparing the Description page for Cadture.

- Problem focus: lead (Pb) and cadmium (Cd) in industrial wastewater.
- We will expand scientific detail after team review.
- We will not report experimental percentages without measured data.

步驟 B3 | 更新 toc（重要，慢慢做）

front matter 的 toc 增加：

```
toc:
  - id: what-should-this-page-contain
    label: What to include
  - id: our-drafting-notes
    label: Drafting notes
```

步驟 B4 | 理解 id 規則（講 5 分）

標題：

```
## Our drafting notes
```

自動 id 大致是：`our-drafting-notes`

（小寫、空白變 `-`、符號常被去掉）

若 toc 的 id 打成 `Our-Drafting-Notes` 可能跳轉失敗。

步驟 B5 | 驗證

- 正文看得到新段落
- 側邊「On this page」點 Drafting notes 會跳轉

5. 完整步驟 C | 追 code（本堂高點，至少 35 分）

C1 | 路由檔

打開 `src/app/description/page.tsx`

找到：

```
<WikiPage slug="description" />
```

在旁邊註解：

```
// slug 必須對應 content/pages/description.md 的檔名
```

C2 | WikiPage

打開 `src/components/WikiPage.tsx`

找到 `getPageBySlug(slug)` 與 `PageHero`、`MarkdownBody`。

用筆在紙上寫：

- frontmatter 進了哪個元件？
- body 進了哪個元件？

C3 | content.ts

打開 `src/lib/content.ts`

找到 `getPageBySlug` :

- 它如何拼出路徑？
- 找不到檔案時會怎樣？

C4 | 小組輪流講（每組 2 分鐘）

不投影答案，逼學生講。

6. 除錯關卡（20 分）

兩人一組，一人出題一人修：

1. 故意刪掉一個 `---` → 修
 2. toc id 打錯 → 修
 3. 在錯誤的 md 檔改字 → 發現並改對檔
-

7. 內容誠信（10 分，不可省）

全班一起念：

1. 英文上站
 2. 無數據不寫假結果
 3. 可寫 `planned` / `literature`
 4. 對齊官方與隊上確認
-

8. 完成檢查表

- ☐ subtitle 有改成功
- ☐ 正文有新增
- ☐ toc 跳轉成功
- ☐ 追 code 三站完成
- ☐ 通過一關除錯

9. 個人作業 #1

認領頁 150–300 字英文 + 手繪資料流（含檔名）。

10. 下一堂

Git：團隊建造的版本基礎設施。

第 5 次 | 工程協作：Git 與團隊建造流程

每次改站都走完整流程——放慢到會看 diff、會寫 message

時數：3 小時

本堂學習目標

- 獨立完成 pull→改→status→diff→add→commit
- 寫出可讀的 commit message
- 能說明衝突如何避免
- 完成誠信情境判斷

0. 分鐘表

分鐘	活動
0—15	為什麼建造需要版本控制
15—70	完整 commit 流程（兩輪）
70—80	休息
80—110	message 工作坊 + 壞例子
110—140	衝突概念與角色扮演
140—165	誠信情境
165—180	里程碑 A

1. 開場故事（放慢）

想像兩人同時改 description.md：

A 寫 Problem，B 刪掉整段改成自己的。沒有 Git 會怎樣？有 Git 可以怎樣？

2. 完整流程（第一輪：跟著做）

2.1 確認目錄

```
cd 你的/iGemSite
git status
```

2.2 若有遠端

```
git pull
```

說明：先同步再改，減少衝突。

2.3 改一個小內容

打開 content/pages/notebook.md，在文末加：

```
## Session 5 note

Date: (today)
We practiced Git workflow for team wiki building.
Next: document real lab notes in English after experiments.
```

2.4 status

```
git status
```

請學生圈出檔名，確認只有預期檔案。

2.5 diff（關鍵，多留時間）

```
git diff
```

老師問：

- + 是什麼？
- 若看到大量刪除，該不該 commit？

2.6 add

```
git add content/pages/notebook.md
```

再 `git status`，檔案應在 staged。

2.7 commit

```
git commit -m "content: add session 5 git practice note to notebook"
```

2.8 驗證

```
git log -1 --oneline  
git status
```

3. 第二輪：自己做（20 分）

再改另一個自己負責的 md 一小段，完整走一次，message 自己想。

4. Commit message 工作坊

好

- content: draft HP listen section without private data
- fix: correct spelling of sequestration on home
- feat: add practice-lab page files

壞 → 改寫練習

把下列改成好 message：

1. update
2. assa
3. 改了很多東西
4. final final 2

5. 衝突角色扮演（25 分）

情境卡：

- 兩人改同一段 subtitle

- 一人 force 心態「以我的為準」

討論：正確流程是什麼？（pull、溝通、合併、誰確認科學內容）

預防規則（寫在黑板）：

1. 一頁同一時段單一主編
2. 改前 pull
3. 小步 commit
4. 大改前通知

6. 誠信情境題（每人寫答案）

1. 官方首頁 90%，文獻也有類似數字，能否寫「我們達成 90%」？為什麼？
2. 指導教師幫改了程式，wiki 要不要寫進 Attributions？
3. 同學照片未經同意能否上 Team 頁？

7. 里程碑 A 繳交

- `git log -5 --oneline` 截圖
- 責任表：頁面 | 人 | 本週摘要
- 誠信三題答案

8. 完成檢查表

- ☐ 兩輪 commit
- ☐ 會看 diff
- ☐ message 合格
- ☐ 誠信題完成

9. 下一堂

結構層：網址如何被「做出來」。

第 6 次 | 結構層：路由系統如何做出網址

App Router 精讀與實驗——放慢到會預測、會破壞、會修復

時數：3 小時

本堂學習目標

- 默寫資料夾與網址規則
- 逐行解釋 page.tsx
- 完成三種安全實驗並還原
- 列出新增路由所需檔案清單

0. 分鐘表

分鐘	活動
0—15	規則精熟遊戲
15—45	精讀 engineering/page.tsx
45—55	休息
55—110	三個實驗
110—140	layout 精讀
140—165	設計題：要哪些檔
165—180	口試抽查

1. 核心規則（背 + 默寫）

```
src/app/<name>/page.tsx  →  /<name>
src/app/page.tsx          →  /
```

遊戲 1 (10 分)

老師報網址，學生寫路徑：

- /team
- /human-practices
- /safety-and-security
- /class/session-06

遊戲 2 (5 分)

老師報路徑，學生寫網址。

2. 精讀 page.tsx (30 分)

打開 `src/app/engineering/page.tsx`：


```
import type { Metadata } from "next";
import WikiPage from "@/components/WikiPage";

export const metadata: Metadata = {
  title: "Engineering",
};

export default function Page() {
  return <WikiPage slug="engineering" />;
}
```

逐行講解稿（老師可照念）

1. 第 1 行 `import type`：帶入型別，執行期可能被擦除，學生知道「TS 提示」即可。
2. 第 2 行 `import WikiPage`：把內容引擎元件拿進來。
3. `metadata`：分頁標題，瀏覽器分頁上可能顯示。
4. `export default function Page`：這個路由的入口。
5. `return <WikiPage slug=... />`：把結構接到系統。
6. `slug` 字串：必須對應 md 檔名。

學生任務

在檔案加註解（可交截圖），四段都標到。

3. 三個實驗（務必還原）

實驗 A | slug 打錯（15 分）

1. 改 `slug="engineeringg"`
2. 存檔、開 `/engineering`
3. 記錄錯誤文字
4. 改回
5. 寫：「錯誤發生在管線第幾步？（讀檔）」

實驗 B | 旁路內容系統（15 分）

暫時改成：

```
export default function Page() {
  return <h1>Routing works without WikiPage</h1>;
}
```

觀察：路由還在，但 md 內容消失。

結論：page 負責「這個網址有元件」；WikiPage 才接內容。

改回 WikiPage。

實驗 C | metadata 標題（10 分）

改 `title: "Engineering (class)"` 看分頁標題，再改回正式名稱。

4. layout.tsx（25 分）

打開 `src/app/layout.tsx`。

找：

- Navbar

- {children}
- Footer

畫圖：

```
RootLayout
├─ SiteLoader (若有)
├─ Navbar
├─ main → children (page)
└─ Footer
```

討論

若沒有 layout，每頁都要自己寫選單，壞處？

加分

打開 `src/app/safety/page.tsx` 看 `redirect`，說明別名路由。

5. 設計題（書面 15 分）

要新增文件頁 `/measurement-notes`，使用 WikiPage 模式，列出需要建立的**完整路徑**與一筆 nav 物件草稿。

6. 完成檢查表

- ☐ 默寫規則
- ☐ 三實驗完成並還原
- ☐ layout 圖完成
- ☐ 設計題交出

7. 作業

口頭 1 分鐘錄音或書面：page.tsx 四段是什麼。

下一堂：nav 資料驅動。

第 7 次 | 結構層：資料驅動的導覽

nav.ts + Navbar 渲染——把物件變成選單（放慢版）

時數：3 小時

本堂學習目標

- 能畫出 navItems → Navbar → 選單 DOM 的關係
- 完成修改 desc、新增 children、除錯三關
- 能解釋資料驅動 UI 的優點

0. 分鐘表

分鐘	活動
0—15	資料驅動概念
15—40	精讀 nav.ts 標註
40—55	精讀 Navbar 如何 map
55—65	休息
65—100	實作改 desc
100—140	實作新增 + 除錯關
140—165	Standard path 討論
165—180	檢查表

1. 概念：不要手寫 20 個連結

壞：Navbar 裡複製 20 次 `<a>`。

好：資料在 `nav.ts`，UI 掃資料生成。

```
navItems —map→ Navbar → 使用者看到的選單
```

2. 精讀 nav.ts (25 分)

步驟

1. 打開 `src/data/nav.ts`
2. 用註解標出：Team / Project / Wet Lab / Dry Lab / HP / Course
3. 抄一筆 children 物件到筆記本，標 label/href/desc

型別（會看即可）

```
children?: { label: string; href: string; desc?: string }[]
```

意思：可選的陣列，元素是物件。

3. 精讀 Navbar (15 分)

打開 `src/components/Navbar.tsx`，搜尋 `navItems`。

回答：

1. 從哪 import ?
2. 哪裡 `.map` ?
3. 有 children 時多做了什麼？

4. 實作 A：改 desc（完整）

1. 選你負責頁
2. 只改 `desc` 英文
3. 存檔
4. 重新整理，打開對應下拉
5. `git diff src/data/nav.ts` 確認無意外

5. 實作 B：新增一筆（完整）

範例（可依現況調整）：

```
{ label: "Course home", href: "/class", desc: "Full build-the-wiki lessons" },
```

注意：

- 上一行結尾逗號
- 字串引號成對
- `href` 以 `/` 開頭

6. 除錯三關（每人輪流）

關	破壞	修復關鍵
1	刪逗號	讀編譯錯誤
2	<code>href</code> 少 <code>/</code>	觀察導向
3	加在錯誤的 children	選單位置不對

每關寫：「我看到的錯誤 → 我怎麼修」

7. Standard Paths（10 分）

可改 label，慎改 href：

- `/attributions`
- `/contribution`
- `/engineering`
- `/human-practices`

8. 完成檢查表

- ☐ 圖：資料→Navbar→選單
- ☐ 改 desc 成功
- ☐ 新增成功
- ☐ 三關除錯紀錄

9. 放慢用的加練（若提前做完）

加練 A | 重畫選單資料（紙上 10 分）

不看電腦，默寫 Project 底下應有的 4 個子項（label + href）。
再打開 nav.ts 對答案。

加練 B | 讀錯訊息（10 分）

老師投影一段錯誤的 nav 片段（少 `] ]`），學生指出第幾行、少了什麼。

加練 C | 口頭解釋（每組 2 分）

「為什麼選單要用資料檔，而不是寫死在 Navbar？」至少講兩個理由。

10. 作業 #2

一句英文 desc；選做：紙上設計三層選單資料。

下一堂：系統引擎 WikiPage——請先複習第 6 次 page.tsx。

第 8 次 | 系統層：元件與內容引擎

整學期理論高點——管線精讀與可觀察實驗（放慢版）

時數：3 小時

本堂學習目標

- 不看稿口述 11 步請求流程
- 四站精讀並互教
- 完成三個可觀察實驗並記錄結論

0. 分鐘表

分鐘	活動
0—20	11 步流程板書（學生抄）
20—30	抽問前 5 步
30—90	四站精讀（輪站）
90—100	休息
100—150	三實驗
150—170	組合圖 + 口試
170—180	作業

1. 11 步流程（抄寫，兩次）

以 `/human-practices`：

1. 瀏覽器請求路徑
2. Next 匹配 `src/app/human-practices/page.tsx`
3. 執行 `Page`
4. 渲染 `WikiPage` 並傳入 `slug`
5. `getPageBySlug` 讀 md 檔
6. `gray-matter` 拆 `frontmatter` / `body`
7. `PageHero` 用 `frontmatter`
8. `ContentLayout` 處理版面與 `toc`
9. `MarkdownBody` 渲染 `body`
10. `layout` 提供 `Navbar/Footer`
11. 完整頁交付

口訣

路找 `page` → `page` 叫 `WikiPage` → `WikiPage` 讀 md → 拆兩包 → 英雄區+正文 → 外殼選單

抽問題庫

- 第 5 步檔案不在？
- `slug` 錯但 `page` 在？
- 沒有 `MarkdownBody`？
- 沒有 `layout`？

2. 四站精讀（每站 12—15 分）

每站任務卡：

站 A content.ts

- 找出拼路徑那一行
- 找出 throw Error
- 一句話：本檔職責

站 B WikiPage.tsx

- props 是什麼
- frontmatter 給誰
- body 給誰

站 C MarkdownBody.tsx

- 哪個套件在轉 md
- h2 為何要自訂

站 D PageHero / ContentLayout

- 有狀態嗎還是純展示
- toc 從哪來

每站結束：換人當小老師講 90 秒。

3. 三個實驗（慢做）

實驗 1 只改 body

md 加 `## Pipeline lab` 一段 → 驗證出現。

結論寫：_____

實驗 2 只改 title frontmatter

大標變。結論：_____

實驗 3 暫時錯 slug 於 page（再修）

對應錯誤。結論：_____

4. 元件組合圖（必畫，10 分）

```
layout
  Navbar
  Page
    WikiPage
      PageHero
      ContentLayout
      MarkdownBody
  Footer
```

5. 完成檢查表

- ☐ 11 步能講
- ☐ 四站互教

- □ 三實驗紀錄
- □ 組合圖

6. 放慢用的「小白板題」（全班 15 分）

老師抽問，答不出就回站重讀 5 分再答：

1. `matter` 之後，`title` 大概在哪個變數裡？
2. 使用者從沒開過 `engineering.md`，為什麼螢幕上會有字？
3. 若 `MarkdownBody` 壞掉，`PageHero` 還會不會在？
4. `Navbar` 的資料是不是也走 `getPageBySlug`？

7. 作業

用語音或文字 1–2 分鐘 說明：`/engineering` 如何從點擊到上屏（需提到至少 5 個檔名）。

下一堂：首頁另一種組裝法——先打開官方首頁複習七段。

第 9 次 | 系統層：首頁如何組裝

home.md 資料 + page.tsx 視圖 + 官方七段——放慢對照

時數：3 小時

本堂學習目標

- 能對照官方與本機逐段檢查
- 能改 YAML 並指到對應 map
- 能比較 WikiPage 模式與首頁模式
- 完成里程碑 B

0. 分鐘表

分鐘	活動
0—20	兩種頁面策略
20—55	官方七段逐段對照
55—65	休息
65—110	改 home.md（允許範圍）
110—150	page.tsx map 導讀與標註
150—170	里程碑 B 整理
170—180	分享 2 組

1. 兩種建造策略（15 分講清楚）

	文件頁	首頁
資料	一篇 pages/*.md	home.md YAML 多段清單
組裝	WikiPage 通用引擎	page.tsx 專用排版
適合	長文、套版結構	固定行銷區塊

問題： 能不能把首頁也改成只有一篇超長 md？可以，但改 UI 區塊順序會變痛。本站選擇資料與視圖分離。

2. 官方對照（放慢 35 分）

左官方右本機，一段一段勾：

段	官方關鍵文字（抄 5—10 字）	本機一致？
Problem 三卡		
三個城市		
Solution 五步名		
Highlights 四數字		
Sustainable 段		
Listen/Integrate/Impact		
Design/Build/Test/Learn		
Team placeholder		

不一致就記下來，課堂討論是否應改回。

3. 改 home.md（完整步驟）

3.1 打開

content/home.md

3.2 允許練習的改動

- locations 的 body 潤飾（不改事實）
- solution 的 body 潤飾（title 建議維持官方五詞）
- teamNote 在隊上同意下更新

3.3 預設禁止

- 擅自改 90% 75% 500+ 12
- 刪除整段 solution

3.4 驗證

存檔 → 重新整理 / → 指到變更區塊給鄰座看。

3.5 YAML 除錯

症狀	原因
首頁空白/錯誤	縮排壞了、--- 不對
某列表少一張卡	少了一個 - title: 區塊
奇怪字元	冒號未加引號

4. page.tsx 導讀（40 分）

步驟 1

打開 src/app/page.tsx

步驟 2

搜尋並在行旁註解：

- h.problems.map → The Problem
- h.locations.map → 城市
- h.solution.map → Our Solution
- h.highlights.map → Highlights
- h.humanPractices.map → HP
- h.engineering.map → Engineering

步驟 3

老師講 .map 與 for 迴圈對照（用第 1 次 JS 例子）。

步驟 4

書面：若 solution 只剩 4 筆，畫面會怎樣？

5. 比較題 (10 分)

用 80—120 字：WikiPage 文件頁 vs 首頁組裝，差在哪？為何首頁要特殊？

6. 里程碑 B

1. 更新了哪兩區 + 確認人姓名
2. 截圖
3. map 標註截圖
4. 比較題答案

7. 完成檢查表

- ☐ 七段對照完成
- ☐ YAML 有合法更新
- ☐ 六個 map 都找到
- ☐ 里程碑 B 交

8. 下一堂

表現層 Tailwind 與設計紀律。

第 10 次 | 表現層：設計系統與 Tailwind

可維護的外觀——放慢拆 class、安全改、RWD

時數：3 小時

本堂學習目標

- 拆解並解釋至少 12 個 class
- 完成兩次安全視覺改動並截圖
- 完成手機與桌機檢查表
- 為負責頁選定 tone 並說明

0. 分鐘表

分鐘	活動
0—15	設計系統是什麼
15—50	拆 class 工作紙
50—60	休息
60—110	安全改動實驗
110—145	RWD 檢查
145—170	tone 作業當堂草稿
170—180	分享

1. 設計系統三層

1. Token（色、圓角觀念） — globals
2. 元件（Button/Card/Hero）
3. 頁面組合

亂改第 3 層還修得回來；亂改第 1 層可能全站爆炸——本堂主攻讀與微調第 2—3 層。

2. 拆 class 工作紙（35 分）

從 `page.tsx` 或 `Card.tsx` 複製兩行長 `className`，填：

class	猜測	查證後
至少 12 格		

必含至少一個 `sm:` 或 `md:` 或 `lg:`。

速查

`mt/mb/p/px/py`、`flex`、`grid`、`gap`、`items-center`、`justify-between`、`text-sm`、`font-bold`、`rounded-`、`border-`、`shadow`、`max-w-*`、`mx-auto`、`hidden`、`sm:block`、`md:grid-cols-3`、`text-primary`

3. 安全改動實驗

實驗 1 間距

某 section `py-14` ↔ `py-10` 或 `py-16`，截圖前後，寫感受。

實驗 2 對齊

某標題 `text-center` 拿掉再加回，討論為何官方區塊常用置中。

還原策略

不喜歡就 `git checkout` 該檔；喜歡就留並 `commit` `style: ...`

4. RWD 完整檢查表

裝置：iPhone 寬 + 桌機 1280

項目	手機	桌機
首頁 Problem		
Solution 五卡		
選單		
文件頁 toc		
課程 /class 可讀		

5. tone 作業當堂完成草稿

在負責頁 md：

```
tone: sky
```

可選：rose / mint / sky / cream / lavender

寫 3 句中文：主題情緒、為何配色合適、有無對比問題。

6. 完成檢查表

- ☐ 12 個 class 表
- ☐ 兩次實驗截圖
- ☐ RWD 表
- ☐ tone 草稿

7. 放慢補充：讀一段真實 class (15 分)

從首頁挑一行，例如（以你檔案實際為準）：

```
mx-auto max-w-7xl px-4 py-14 sm:px-6 lg:px-8
```

逐個解釋：

片段	意思
<code>mx-auto</code>	水平置中
<code>max-w-7xl</code>	最大寬度
<code>px-4</code>	左右內距
<code>py-14</code>	上下內距
<code>sm:px-6</code>	較寬螢幕左右內距加大
<code>lg:px-8</code>	更寬再加大

練習：學生自己再拆一行，寫進筆記本。

8. 下一堂

總裝新頁——學期建造驗收。請先複習 session-06 路由 + session-07 nav + session-04 md。

第 11 次 | 總裝：從零組出完整頁面

內容+結構+系統串接——詳細步驟與除錯劇本

時數：3 小時

本堂學習目標

- 獨立完成 md + page.tsx + nav
- 通過除錯三關
- 書面解釋三步驟對應三層

0. 分鐘表

分鐘	活動
0—15	三層對照複習
15—50	步驟① md
50—60	休息
60—100	步驟② page
100—130	步驟③ nav
130—165	除錯關
165—180	書面三層解釋

1. 開始前口頭複習（10 分）

1. 只有 md ?
2. 只有 page ?
3. 有 page+md 沒 nav ?

2. 步驟① 內容層（詳細）

2.1 建立檔案

```
content/pages/practice-lab.md
```

2.2 最小可用內容

```
---
title: Practice Lab
eyebrow: Course
subtitle: Built in session 11 to prove we can assemble a page.
tone: mint
toc:
  - id: why-this-page
    label: Why
  - id: how-it-is-built
    label: How built
---

## Why this page

This page is a coursework practice page for VIS team wiki training.

## How it is built

1. Markdown file in content/pages
2. Route file in src/app/practice-lab/page.tsx
3. Navigation entry in src/data/nav.ts

The WikiPage component loads this Markdown by slug.
```

2.3 驗證內容檔本身

此時可不開瀏覽器，但檔案必須存在且 YAML 合法。

3. 步驟② 路由（詳細）

3.1 建資料夾

```
src/app/practice-lab/
```

3.2 建 page.tsx

```
import type { Metadata } from "next";
import WikiPage from "@components/WikiPage";

export const metadata: Metadata = {
  title: "Practice Lab",
};

export default function Page() {
  return <WikiPage slug="practice-lab" />;
}
```

3.3 驗證

```
http://localhost:3000/practice-lab
```

3.4 若失敗

現象	查
Missing content file	slug/檔名
404	資料夾名或 dev 未重載
編譯錯	import 路徑、括號

4. 步驟③ nav (詳細)

在 `nav.ts` 合適 children 加：

```
{ label: "Practice Lab", href: "/practice-lab", desc: "Session 11 assembly practice" },
```

驗證：選單可點、手機選單也可。

5. 除錯關 (必做)

交換電腦破壞：

1. slug 少一個字母
2. 檔名與資料夾不一致
3. nav href 錯

修復者需向破壞者解說。

6. 書面題 (交)

用表格：

步驟	檔案路徑	對應層	若省略會怎樣
1		內容	
2			
3			

7. 完成檢查表 (硬性)

- ☐ 三步驟成功
- ☐ 除錯關通過
- ☐ 書面題完整
- ☐ 無假科學宣稱

8. 作業

英文精修 How it is built；選做 code review 他組。

下一堂：產品品質。

第 12 次 | 產品品質：媒體、寫作與可信介面

做得出來之後，還要做得可信——放慢寫作與互改

時數：3 小時

本堂學習目標

- 完成圖片上頁流程
- 完成兩段英文並經互改
- 用品質量表自評到可給外人看

0. 分鐘表

分鐘	活動
0—15	可發布標準
15—50	圖片流程
50—60	休息
60—130	英文工坊（寫+互改+上站）
130—160	品質量表
160—180	分享佳句

1. 可發布標準（一起念）

1. 結構清楚
2. 英文可讀
3. 圖/表有說明
4. 有限制與下一步
5. 無假數據
6. 風格不跑版

2. 圖片完整步驟

1. 選圖、壓縮
2. 英文檔名
3. 放到 `public/images/`
4. md：

```
## Figure

! [Describe what the viewer should see] (/images/your-file.png)

**Figure 1.** Caption. Say if coursework-only.
```

5. 檢查 alt 是否有用
6. 窄螢幕看圖是否過大

討論（5 分）

正式 iGEM 站圖片托管限制——為何不能隨便外連相簿？

3. 英文工坊（放慢）

3.1 句型紙（印發）

Problem / Method / DBTL / HP / Uncertainty 各 2 句模板（見舊教材句型，本堂展開寫）。

3.2 個人寫作 20 分

選一：Description 問題段 或 Engineering 半輪 DBTL 或 HP 一則 Listen-Integrate。

字數 150—250。

3.3 互改表 20 分

項目	原句問題	修改建議
主詞不清		
時態像已完成但其實沒做		
太絕對		
專有名詞不統一		

3.4 上站 15 分

貼進 md，重新整理，給老師看「互改前/後」各一句。

4. 品質量表自評

向度	1	2	3	4	我的分數
架構					
技術					
英文					
誠實					
風格					

低於 3 的向度寫改進行動。

5. 作業 #4

定稿 200 字上站。

6. 放慢：句型擴寫紙（可印）

Problem

[Topic] is important because [impact on people/environment].
Current methods often [limitation].
Our project, Cadture, focuses on [Pb/Cd + approach in one line].

DBTL

In this cycle we targeted [bottleneck].
We designed [idea] because [reason].
We built / plan to build [construct or protocol].
We tested / will test [metric] with [control].
We learned that [lesson], so next we will [change].

HP

We spoke with [role], who said [point].
We integrated this by [design/safety/education change].
This matters because [impact].

每位學生至少完整填一張再上站。

7. 下一堂

build 與交付檢查——工程師怎麼交付。

第 13 次 | 工程交付：Build、檢查與上線概念

本機可以還不夠——完整檢查與 *build* 放慢教學

時數：3 小時

本堂學習目標

- 逐項完成品質檢查並實點連結
- 獨立跑 `npm run build` 並記錄
- 能說明 `dev/build/freeze` 差異
- 交出里程碑 C

0. 分鐘表

分鐘	活動
0–15	交付金字塔
15–70	品質檢查實點
70–80	休息
80–130	build 教學與修復
130–160	iGEM/freeze 概念
160–180	里程碑 C 彙整

1. 交付金字塔

可展示給外人
↑
build 過 + 連結對 + 內容實
↑
功能做得出

2. 品質檢查（詳細流程）

使用 `docs/curriculum/handouts/13-quality-checklist.md`。

2.1 內容（每項寫證據）

不是只勾，要寫「我看到的網址/句子」。

2.2 連結實點清單

至少點：

- 首頁
- `description / engineering / human-practices / contribution / team / attributions`
- `class` 課表
- 自己新增頁

壞掉的寫進待修。

2.3 路徑穩定

確認未把 `engineering` 等改名。

3. build 完整教學

3.1 指令

```
cd 你的/iGemSite
npm run build
```

3.2 成功長怎樣

出現編譯成功、靜態頁生成，最後無 Failed。截圖。

3.3 失敗怎麼讀

1. 找到第一個 `Error`
2. 檔案路徑
3. 行號
4. 訊息關鍵字

3.4 常見錯誤對照

關鍵字	可能原因	先查
Missing content file	slug/md	page 與 md
Unexpected token	nav 語法	逗號括號
Type error	TS	問老師
Module not found	import 路徑	@/ 路徑

3.5 修復循環

修 → 再 build → 直到過或寫卡關報告。

4. 環境差異（講慢）

環境	用途	限制
dev	開發	較寬鬆
build	交付檢查	較嚴
官方 wiki	競賽基準/freeze	規則最多

讀 `docs/DEPLOY_T0_IGEM.md` 重點：外連、字型、圖、CC BY、Freeze。

口頭題

1. 本機可以為何還要 build？
2. Freeze 後還能大改 UI 嗎？
3. 外連 Google 字型可能有何問題？

5. 里程碑 C

1. 檢查表（含證據）
2. build 截圖
3. 待修 ≥ 3 （人+期限）
4. 一句話風險

6. 完成檢查表

- ☐ 實點連結
- ☐ build 有跑
- ☐ 概念三題能答
- ☐ 里程碑 C 交

7. 下一堂

發表：用架構證明你會做這個站。

第 14 次 | 成果發表：證明你會做這個網站

完整發表流程、質問庫、維護公約——可直接照表執行

時數：3 小時

本堂學習目標

- 依強制大綱完成發表
- 回答架構質問
- 簽維護公約
- 完成個人反思

0. 當日時間表

分鐘	活動
0—20	最終技術檢查
20—100	發表（含 Q&A）
100—110	休息
110—140	互評
140—165	教師回饋 + 維護公約
165—180	反思單

1. 開場前技術檢查（詳細）

- ☐ `npm run dev` 或展示環境可開
- ☐ 首頁七段還在
- ☐ 核心頁無 404
- ☐ `practice-lab` 或新增頁可開
- ☐ `build` 結果已知
- ☐ 投影縮放 125% 可讀
- ☐ 無私人帳號畫面

2. 強制發表大綱（6—8 分）——請逐段計時

0:00—0:30 開場

組員、分工

0:30—1:30 產品

Capture 是什麼；為何對齊官方站

1:30—3:30 架構 ★

- 三層模型
- 口述資料流（可看自己的圖，不可啞口）
- 首頁模式 vs WikiPage 模式

3:30—5:30 實作證據

- 現場或錄影：改 md
- 展示 nav 或新增頁
- build 結果

5:30—6:30 誠信

一處 planned / 未完成標示

6:30—7:30 維護

誰繼續更新

7:30—8:00 結語

3. 質問庫（老師或同儕）

1. 只有 md 沒 page？
2. slug 做什麼？
3. nav 如何變 DOM？
4. 為何不隨便改 90%？
5. layout 的 children？
6. gray-matter 拆出什麼？
7. `.map` 在首頁的作用？
8. Standard path 是什麼？

4. 互評

用 docs/curriculum/handouts/14-presentation-rubric.md

建議：架構理解 < 3 則總評不得 > 3

5. 維護公約（當場填）

工作	負責人	頻率	簽名
首頁文案			
Description			
Engineering			
HP			
Notebook		每週	
Results		有數據	
/class 教材	老師/助教		

6. 個人反思單

1. 三個最有用技能
2. 最怕的一步與為何
3. 若重來第 8 堂會怎樣
4. 誠信一例

5. 我是否達到「會做站」而不只是「會改字」？為什麼？

7. 課程結束宣言（可齊念）

我們不只會改 *Markdown*。
我們理解內容、結構與系統。
我們用誠實的內容，維護 *Cadtare Team Wiki*。

8. 之後

繼續用 `/class` 當手冊；有數據再填 Results；大改前先 build。

四、講義（可撕下 / 單獨影印）

第 1 次 | 環境架設檢查表

姓名：_____ 組別：_____

安裝

- ☐ 已安裝 VS Code
- ☐ 已安裝 Node.js (終端機執行 `node -v` 有版本號)
- ☐ 已取得專案資料夾

啟動

在專案根目錄執行：

```
npm install
npm run dev
```

- ☐ 終端機沒有紅色錯誤
- ☐ 瀏覽器打開 <http://localhost:3000> 看得到 Cadture

導覽確認 (在網站上點過)

- ☐ Description
- ☐ Engineering
- ☐ Human Practices
- ☐ Team
- ☐ 任一 Wet Lab 頁

今日一句話

這個 wiki 最重要的讀者是：_____

Markdown 速查 (第 4 次)

```
# 最大標題 (頁面通常用 front matter 的 title)
## 次標題
### 小標題
```

一般段落這樣寫。

```
**粗體**  *斜體*
```

- 項目一
- 項目二

1. 第一
2. 第二

[連結文字] (<https://2026.igem.wiki/vis/>)

![說明文字] (/images/example.png)

> 重要提示或引用

欄位 A 欄位 B
----- -----
內容 內容

本站頁面檔位置

content/pages/頁名.md

Front matter 範例

```
---
title: Description
eyebrow: Project
subtitle: Short English line
tone: rose
toc:
  - id: what-should-this-page-contain
    label: What to include
---
```

What Should this Page Contain? 的 id 會變成 what-should-this-page-contain 。

第 6 次 | 你會的 JS → 本站 TSX

對照

```
// 你可能寫過
function greet(name) {
  return "Hello " + name;
}
```

```
// 本站 page 很像「回傳畫面的 function」
import WikiPage from "@components/WikiPage";

export default function Page() {
  return <WikiPage slug="description" />;
}
```

JS	這裡
function Page()	元件
return ...	回傳 JSX（長得像 HTML）
"description"	傳給元件的資料（props）
import ...	使用別的檔案

網址規則

資料夾 `src/app/engineering/` + 檔案 `page.tsx`

→ 網址 `/engineering`

小練習

1. `/team` 對應哪個資料夾？
2. `slug="team"` 寫成 `slug="teams"` 會怎樣？
3. 長文通常改 `content/pages/....md` 還是 `page.tsx` ？
(1) `src/app/team/` (2) 找不到對應 md 或內容錯誤 (3) md

第 8 次 | 內容如何變成網頁

把步驟背起來，考試 / 發表會用得上：

1. 你編輯 `content/pages/engineering.md`
2. 函式 `getPageBySlug("engineering")` (`src/lib/content.ts`)
3. 元件 `WikiPage` (讀 front matter + 正文)
4. 元件 `MarkdownBody` (Markdown → HTML)
5. 版面 `PageHero + ContentLayout`
6. 路由 `src/app/engineering/page.tsx` → 網址 `/engineering`

用 JS trace 一次

```
// page.tsx 裡大致是：  
<WikiPage slug="engineering" />  
//           ↑ 像函式參數  
  
// WikiPage 裡會：  
getPageBySlug(slug) // 像 call function(slug)  
// 得到 { frontmatter, body } // 像得到一個物件
```

小測驗

1. 只改英文段落，應改哪個資料夾？
2. 網址 `/team` 對應哪個資料夾？
3. 全站上方選單主要改哪個檔？
4. `slug` 比較像：迴圈 / 函式參數 / CSS 顏色？

答：1) `content/` 2) `src/app/team/` 3) `src/data/nav.ts` 4) 函式參數

第 11 次 | 新增一頁三步驟

1. 內容

新增 `content/pages/my-page.md`：

```
---
title: My Page
eyebrow: Project
subtitle: One English sentence.
tone: mint
---

## Overview

Write English here.
```

2. 路由

新增 `src/app/my-page/page.tsx` (可複製 `description` 那份)：

```
import type { Metadata } from "next";
import WikiPage from "@components/WikiPage";

export const metadata: Metadata = {
  title: "My Page",
};

export default function Page() {
  return <WikiPage slug="my-page" />;
}
```

注意：`slug` 必須和檔名 `my-page.md` 相同。

3. 導覽

在 `src/data/nav.ts` 合適的 `children` 裡加入：

```
{ label: "My Page", href: "/my-page", desc: "Short English desc" },
```

檢查

- ☐ <http://localhost:3000/my-page> 打得開
- ☐ 選單點得到
- ☐ 英文沒有未確認的假數據

第 13 次 | 品質與競賽意識檢查表

組別：_____ 成員：_____

內容

- ☐ 英文為主，專案名稱 Cadture 一致
- ☐ 沒有未確認的實驗數據
- ☐ 各核心頁有負責人姓名
- ☐ 圖片有說明

負責人頁面登記：

頁面	負責人
Description	
Engineering	
Human Practices	
Contribution	
Team	
其他：	

結構

- ☐ 導覽連結都有效
- ☐ 沒有亂改 Standard 路徑
- ☐ 新增頁具備 md + page + nav

技術

- ☐ `npm run dev` 正常
- ☐ 嘗試過 `npm run build` (結果：通過 / 有錯誤：_____)
- ☐ 手機寬度粗看可讀

待修清單 (至少 3 項)

- 1.
- 2.
- 3.

第 14 次 | 發表會互評表

評分組別：_____ 被評組別：_____

每項 1–4 分（4 最高）

項目	分數	一句話回饋
架構說明清楚		
實作證據足夠		
設計風格一致		
內容誠實可信		
分工合作		
總分		

我想學他們的一點：

我建議他們下週優先改：

五、給同學的提醒

1. 改 `content/` 裡的檔 → 存檔 → 瀏覽器重新整理
2. 不會的先看錯誤訊息**第一行**，再舉手
3. Wiki 科學內容以隊上確認與官方站為準，**不編造**
4. 完整說明也在網站 `/class`，本講義方便離線與勾選